

Week 06 Summary: Object Design Refinement

Case Study: The Mark IV Special Coffee Maker (Heuristics and Coffee)

1. Overview of Week 06 Learning Objectives

This week focuses on **Object Design (Part II) – Design Refinement and Better Design**. Based on Chapter 11, "Heuristics and Coffee", the module teaches how to identify and rectify poor architecture choices, properly separate concerns, apply the **Dependency Inversion Principle (DIP)**, and abstract high-level business logic away from hardware-dependent low-level execution details.

2. Problem Context: The Mark IV Special Coffee Maker

The assignment tasks us with designing control software for a real-time, single-threaded embedded system with specific physical hardware constraints:

- **Boiler Switch / Sensor:** Recognizes two states: `BOILER_EMPTY` or `BOILER_NOT_EMPTY`.
- **Warmer Plate Sensor:** Recognizes three states: `WARMER_EMPTY`, `POT_EMPTY`, or `POT_NOT_EMPTY`.
- **Brew Button:** Triggers the brewing cycle and automatically resets. Includes an indicator light to signal when the cycle is complete.
- **Pressure-relief Valve:** Closes to build up pressure and deliver boiling water to the filter; opens to vent steam and stop the water flow immediately.

3. Analysis of the "Hideous Solution" (Common Design Flaws)

A naive approach often yields a hyper-concrete, procedurally driven design disguised as object-oriented. Below is the detailed breakdown of these critical flaws:

Design Flaw / Error	Description & Consequences
Missing Methods	The class diagram is strictly a collection of physical components (nouns) without any behaviors (methods). True object-oriented design must prioritize the active responsibilities and behaviors of the system over passive structural models.
Vapor Classes	Classes such as <code>Light</code> or <code>Button</code> carry no local attributes or internal states. They serve merely as "call translators" that pass execution directly to the raw underlying hardware API. They yield zero architectural value and can be removed entirely.
Imaginary Abstraction	Creating base classes or interfaces (e.g., <code>Sensor</code> or <code>Heater</code>) that are never utilized by clients, or whose contractual interfaces are poorly defined. For instance, a generic <code>sense()</code> method whose dynamic return type completely alters based on the child component breaks clean contracts.
God Classes (The All-Powerful Class)	Although the structure looks modular on paper, all processing logic, branching, and intelligence are centralized within a single class (<code>CoffeeMaker</code>). This leaves the surrounding classes as passive dummies, reducing the architecture back to procedural programming.

4. Refined Clean Object-Oriented Solution

The Abstract Nature of Coffee Making

To eliminate hardware coupling, the refined design divides the primary high-level policies into three autonomous boundaries of responsibility:

1. **HotWaterSource**: Responsible for boiling the water and pumping it through the filter.
2. **ContainmentVessel**: Responsible for holding the brewed coffee, keeping it warm, and sensing its volume.
3. **UserInterface**: Responsible for receiving human inputs and rendering system status back to the user.

Resolving "Crossed Wires"

A common misstep is modeling UML association arrows based on the physical flow of physical water or coffee. In software architecture, association denotes **communication and message passing**. Therefore, the `ContainmentVessel` maintains an association toward the `HotWaterSource` to send control signals (e.g., `Pause`, `Resume`, `Stop`) when the pot is moved or full.

Applying the Dependency Inversion Principle (DIP)

Core Concept: High-level policy classes (`HotWaterSource`, `ContainmentVessel`, `UserInterface`) must remain abstract and completely agnostic of specific Mark IV hardware APIs.

All business rules are kept inside clean interfaces. The specific hardware implementations are pushed out to boundary concrete classes: `M4UserInterface`, `M4HotWaterSource`, and `M4ContainmentVessel`. This architectural inversion allows the core brewing engine to be completely reused across entirely different platforms (e.g., automated tea brewers, vending machines, or commercial soup kettles) without modifying a single line of high-level code.

5. Single-Threaded Control via Polling

Because the target embedded hardware lacks a multi-threaded runtime or hardware interrupt hooks, asynchronous states must be caught manually. The solution employs a `Pollable` interface exposing a `poll()` function. The `main()` system runtime executes an infinite loop, constantly invoking `poll()` on the M4 concrete boundaries to track events and respond with zero latency.

6. Architectural Trade-offs & Over-design (OOverkill)

- Implementing this small application via a **Finite State Machine (FSM)** requires only 7 clear states, 18 deterministic transitions, and less than a single page of clean procedural code.
- The clean, production-grade **Object-Oriented Design** (including test stubs) expands into over 5 full pages of boilerplate and polymorphic code.

Key Takeaway: For ultra-simple applications, the overhead of extensive OOD can outweigh its benefits (Object Overkill). However, practicing this strict separation of concerns and rigorous dependency management is essential training for engineering massive, enterprise-scale software architectures where requirements shift constantly.

Document compiled for academic reference — Focus: Clean Code, OOD Heuristics, and SOLID Principles.